```
#Experiment 1: Setting Up the Environment and Preprocessing the Data
import pandas as pd
import numpy as np
```

```
from sklearn.datasets import load_iris

iris = load_iris()

df = pd.DataFrame(iris.data, columns=iris.feature_names)
df["Species"] = iris.target

df.head()
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	Species
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0

Next steps: Generate code with df | New interactive sheet

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   sepal length (cm)  150 non-null    float64
 1   sepal width (cm)   150 non-null    float64
 2   petal length (cm)  150 non-null    float64
 3   petal width (cm)   150 non-null    float64
 4   Species            150 non-null    int64
dtypes: float64(4), int64(1)
memory usage: 6.0 KB
```

```
df.describe()
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	Species
count	150.000000	150.000000	150.000000	150.000000	150.000000
mean	5.843333	3.057333	3.758000	1.199333	1.000000
std	0.828066	0.435866	1.765298	0.762238	0.819232
min	4.300000	2.000000	1.000000	0.100000	0.000000
25%	5.100000	2.800000	1.600000	0.300000	0.000000
50%	5.800000	3.000000	4.350000	1.300000	1.000000
75%	6.400000	3.300000	5.100000	1.800000	2.000000
max	7.900000	4.400000	6.900000	2.500000	2.000000

```
df.isnull().sum()
```

	0
sepal length (cm)	0
sepal width (cm)	0
petal length (cm)	0
petal width (cm)	0
Species	0

dtype: int64

```
# Add artificial missing values

df.loc[5:10, 'sepal length (cm)'] = np.nan

df.isnull().sum()
```

	0
sepal length (cm)	6
sepal width (cm)	0
petal length (cm)	0
petal width (cm)	0
Species	0

dtype: int64

```
# Fill missing values

df.fillna(df.mean(numeric_only=True), inplace=True)

df.isnull().sum()
```

	0
sepal length (cm)	0
sepal width (cm)	0
petal length (cm)	0
petal width (cm)	0
Species	0

dtype: int64

```
# Label Encoding

from sklearn.preprocessing import LabelEncoder

encoder = LabelEncoder()

df['Species'] = encoder.fit_transform(df['Species'])

df.head()
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	Species
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0

Next steps: Generate code with df | New interactive sheet

```
# Feature Scaling

from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

X = scaler.fit_transform(df.iloc[:,:4])

scaled_df = pd.DataFrame(X, columns=df.columns[:4])

scaled_df.head()
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
0	-0.973943	1.019004	-1.340227	-1.315444
1	-1.223494	-0.131979	-1.340227	-1.315444
2	-1.473046	0.328414	-1.397064	-1.315444
3	-1.597821	0.098217	-1.283389	-1.315444
4	-1.098719	1.249201	-1.340227	-1.315444

Next steps: Generate code with `scaled_df` New interactive sheet

```
# Train Test Split

from sklearn.model_selection import train_test_split

X_train,X_test,y_train,y_test=train_test_split(
    X,
    df['Species'],
    test_size=0.2,
    random_state=42
)

print("Training Samples:",len(X_train))
print("Testing Samples:",len(X_test))
```

```
Training Samples: 120
Testing Samples: 30
```